Chapter 17

Large language models

Natural language processing is another area where pretraining models are particularly successful. The basic object we would like to learn is the distribution of language: which sentences are plausible, how a piece of text is likely to continue. If a model can assign probabilities to text sequences and sample continuations from conditional distributions, then many tasks can be phrased as conditional generation from a prompt. This is why language modeling, and in particular next-token prediction, is a useful pretraining objective for language data.

17.1 Tokenization

Before defining the language-modeling objective, we need to specify how raw text is represented as model inputs. A language model does not read a sentence as one raw string. Instead, the text is first broken into small pieces called *tokens*. A token is simply a unit of text that the model treats as one symbol: it may be a whole word like `hello`, part of a word like `un`, punctuation, whitespace, a byte in ASCII encoding, or a special marker such as an end-of-text token. A *tokenizer* is the preprocessing rule that converts a string into a sequence of tokens, which we identify with discrete ids

$$x = (x_1, \dots, x_T), \qquad x_t \in V,$$

where V is the vocabulary, meaning the set of all tokens the tokenizer can output. For example, the sentence `A dog runs.` might be represented as tokens roughly corresponding to `A`, `dog`, `runs`, and `.`; a rarer word such as `unhappiness` might be split into pieces such as `un`, `h`, and `appiness`.

Modern language models usually use *subword* tokenization, often based on byte-pair encoding (BPE) [Sennrich et al., 2016, Kudo and Richardson,

2018]. Subword tokenization is a compromise between two simpler choices: character-level tokenization has a small vocabulary but makes every sentence long, while word-level tokenization makes common sentences short but struggles with rare or new words. For example, if a word-level vocabulary does not contain a rare biomedical term such as `glioblastoma`, or a newly coined form such as `LLMification`, it may need to map each whole string to an unknown token. A subword tokenizer can instead encode them using reusable pieces, such as `glio`+`blast`+`oma` or `LL`+`M`+`ification`, up to the exact learned segmentation. The intuition behind byte-pair encoding is to start from very small pieces, such as bytes or characters, and repeatedly add a new token for an adjacent pair that occurs often in the text corpus. After many such merges, common strings such as `ing` or `tion`, and oftentimes whole common words, can be represented by one token, while rare words can still be represented by smaller pieces. Once the tokenizer is fixed, pretraining is formulated over token sequences rather than raw text; the choice of tokenizer affects the vocabulary size $|V|$ and the resulting sequence length T. The vocabulary size $|V|$ can be on the order of 10^5; for example, Qwen3.5 uses a vocabulary of 248,320 tokens.

17.2 Autoregressive models and next-token prediction loss

We typically operate on the sequence level in the pre-training, that is, each "example" is a sequence. A document (or a concatenation of multiple documents) will be first tokenized into a sequence of tokens, denoted by $x = (x_1, \cdots, x_T)$, where each x_i is a token that belongs to the vocabulary V. When convenient, we identify the tokens in V with the indices $\{1, \ldots, |V|\}$.

A language model is a probabilistic model representing the probability of the sequence, denoted by $p(x_1, \cdots, x_T)$. This probability distribution is very complex because its support is V^T with size $|V|^T$. The vocabulary size $|V|$ can be on the order of 10^5; for example, Qwen3.5 uses a vocabulary of 248,320 tokens. And T can easily be on the order of 10K or as large as 1M, and thus $|V|^T$ is an astronomical number. Thus, instead of modeling the distribution of a sequence itself, we apply the chain rule of conditional probability to decompose it as follows:

$$p(x_1, \cdots, x_T) = p(x_1)p(x_2|x_1)\cdots p(x_T|x_1, \cdots, x_{T-1}). \tag{17.1}$$

Now the support size of each of the conditional probability $p(x_t|x_1, \cdots, x_{t-1})$ is $|V|$.

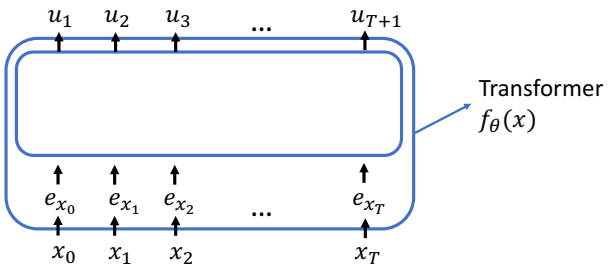


Figure 17.1: The inputs and outputs of a Transformer model.

We will model the conditional probability $p(x_t|x_1, \cdots, x_{t-1})$ as a function of $x_1, \ldots, x_{t-1}$ parameterized by some parameter θ.

A parameterized model takes in numerical inputs and therefore we first introduce embeddings or representations of the tokens. Let $e_i \in \mathbb{R}^{1\times d}$ be the row-vector embedding of the i-th token. Let

$$E = \begin{bmatrix} e_1 \\ e_2 \\ \vdots \\ e_{|V|} \end{bmatrix} \in \mathbb{R}^{|V|\times d} \tag{17.2}$$

be the collection of the embeddings..

The most commonly used models are autoregressive versions of the Transformer [Vaswani et al., 2017] and its variants. In this section, we will introduce the input-output interface of a Transformer, but treat the intermediate computation in the Transformer as a black box, while in Section 17.3 we will discuss the details of the Transformer and other variants.

As shown in Figure 17.1, given a tokenized document $(x_1, \cdots, x_T)$, we first translate the sequence of discrete variables into a sequence of corresponding token embeddings $(e_{x_1}, \cdots, e_{x_T})$.[1] We also introduce a fixed special token $x_0 = \perp$ in the vocabulary with corresponding embedding $e_\perp$ to mark the beginning of a document. Then, the token embeddings are passed into a Transformer model, which takes in a sequence of vectors $(e_{x_0}, e_{x_1}, \cdots, e_{x_T})$

[1]The mapping from raw text to $(x_1, \ldots, x_T)$ is determined by the tokenizer introduced in Section 17.1. The treatment of special boundary tokens is also model- and tokenizer-dependent: some models prepend a beginning-of-sequence token, while others pack documents into long streams and use end-of-text tokens after each document, without inserting a separate beginning token. We also omit positional encodings from this input-output interface; practical Transformers add positional information to distinguish token order, and we will add this discussion in a future revision.

and outputs a sequence of vectors $(u_1, u_2, \cdots, u_{T+1})$, where $u_t \in \mathbb{R}^{|V|}$ will be interpreted as the logits for the probability distribution of the next token. Here we use the autoregressive version of the Transformers which by design ensures u_t only depends on $x_1, \cdots, x_{t-1}$ (see Section 17.3 for the definition of Transformers, and note that this property does not hold in masked language models [Devlin et al., 2019] where the losses are also different.) We view the whole mapping from x's to u's as a black box in this subsection and call it a Transformer, denoted by f_θ, where θ includes both the parameters in the Transformer and the input embeddings. We write $u_t = f_\theta(x_0, x_1, \ldots, x_{t-1})$ where f_θ denotes the mapping from the input to the outputs.

The conditional probability $p_\theta(x_t|x_1, \cdots, x_{t-1})$ is the softmax of the logits:

$$\begin{bmatrix} p_\theta(x_t = 1|x_1 \cdots, x_{t-1}) \\ p_\theta(x_t = 2|x_1 \cdots, x_{t-1}) \\ \vdots \\ p_\theta(x_t = |V||x_1 \cdots, x_{t-1}) \end{bmatrix} = \text{softmax}(f_\theta(x_0, \ldots, x_{t-1})) \in \mathbb{R}^{|V|} \tag{17.3}$$

Autoregressive generation / sampling / inference. Given an autoregressive Transformer, we can sample text from it sequentially. Given a prefix $x_1, \ldots, x_t$, we generate a token completion $x_{t+1}, \ldots, x_T$ sequentially using the conditional distribution.

$$x_{t+1} \sim \text{softmax}(f_\theta(x_0, x_1, \cdots, x_t)) \tag{17.4}$$
$$x_{t+2} \sim \text{softmax}(f_\theta(x_0, x_1, \cdots, x_{t+1})) \tag{17.5}$$
$$\ldots \tag{17.6}$$
$$x_T \sim \text{softmax}(f_\theta(x_0, x_1, \cdots, x_{T-1}))\,. \tag{17.7}$$

Note that each generated token is used as the input to the model when generating the following tokens. In practice, people often introduce a parameter $\tau > 0$ named *temperature* to further adjust the entropy/sharpness of the generated distribution,

$$x_{t+1} \sim \text{softmax}(f_\theta(x_0, x_1, \cdots, x_t)/\tau) \tag{17.8}$$
$$x_{t+2} \sim \text{softmax}(f_\theta(x_0, x_1, \cdots, x_{t+1})/\tau) \tag{17.9}$$
$$\ldots \tag{17.10}$$
$$x_T \sim \text{softmax}(f_\theta(x_0, x_1, \cdots, x_{T-1})/\tau)\,. \tag{17.11}$$

When $\tau = 1$, the text is sampled from the original conditional probability defined by the model. With a decreasing τ, the generated text gradually

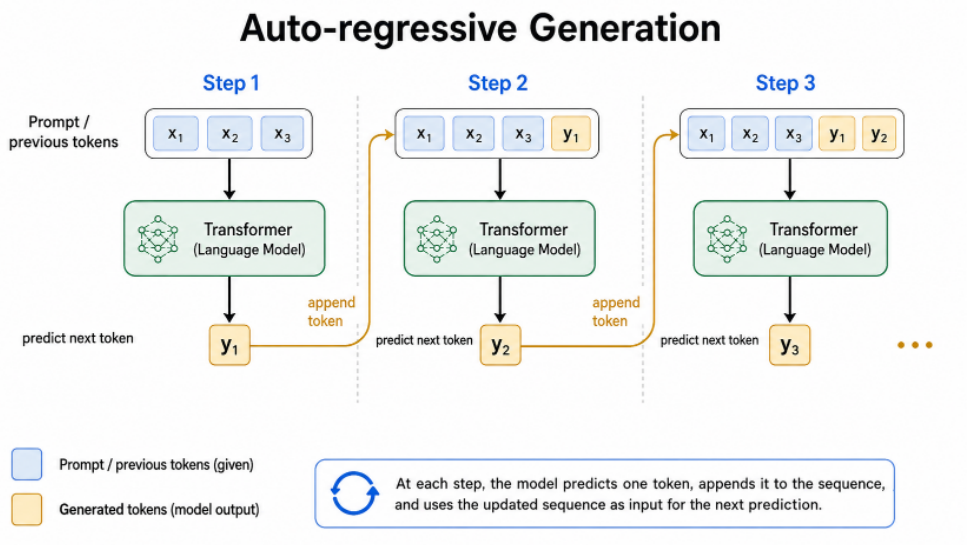


Figure 17.2: Autoregressive generation feeds each newly generated token back into the prefix before generating the next token.

becomes more "deterministic". $\tau \to 0$ reduces to greedy decoding, where we generate the most probable next token from the conditional probability.

Two other common decoding heuristics are *top-k sampling* and *top-p sampling*, also called nucleus sampling. Instead of sampling from the full vocabulary distribution, top-k sampling keeps only the k tokens with largest probability and renormalizes the distribution over this smaller set. Top-p sampling keeps the smallest set of highest-probability tokens whose total probability is at least p, and renormalizes over that set. These methods are inference-time choices rather than changes to the parameterized probability distribution of the resulting training loss; they are often combined with temperature to avoid sampling from the long tail of very unlikely tokens while still allowing non-greedy generations. For example, a common choice is top-p sampling with $p = 0.9$ together with a temperature in the range 0.7–1.0, which avoids sampling from the long tail of very unlikely tokens while still allowing non-greedy generations.

Pre-training auto-regressive LLMs The pretraining loss for training an auto-regressive Transformer θ is simply the maximum likelihood estimator, that is, minimizing the negative log-likelihood of seeing the data under the probabilistic model defined by θ, which is the cross-entropy loss on the logits. Note for convenience people normalized by the sequence length T. For one

sequence, it writes

$$\begin{aligned}
\text{loss}(x_1, \ldots, x_T; \theta) &= -\frac{1}{T} \log p_\theta(x_1, \ldots, x_T) && (17.12)\\
&= \frac{1}{T} \sum_{t=1}^{T} -\log(p_\theta(x_t | x_0, \ldots, x_{t-1})) && (17.13)\\
&= \frac{1}{T} \sum_{t=1}^{T} \ell_{\text{ce}}(f_\theta(x_0, x_1, \cdots, x_{t-1}), x_t) \\
&= \frac{1}{T} \sum_{t=1}^{T} -\log(\text{softmax}(f_\theta(x_0, x_1, \cdots, x_{t-1}))_{x_t}) \,.
\end{aligned}$$

In practice there are a larger number of training sequences, and the resulting training loss is the average of the loss over all sequences. A common optimizer for modern language-model training is AdamW [Loshchilov and Hutter, 2019], which takes a mini-batch of these sequences, computes the gradient, and applies the update rule.

17.3 Transformer architecture

Transformers, the dominant model architecture for language modeling, are composed of multiple blocks of multi-head self-attention and MLPs. The MLPs typically use GELU activations or SwiGLU activations as we mentioned in Section 7.3. In this section, we will introduce self-attention in details. We start with single-head self-attention, introduce multi-head self-attention , and its two variants: multi-query attention and grouped-query attention. We will then describe how multi-head self-attention and MLPs are connected as building blocks inside a Transformer.

As the input to the language model is a sequence of tokens, the model needs to gather information from all the previous tokens to predict the next token. This is not easily achievable in the standard feedforward neural networks, as the input dimension will be changing with T. The self-attention mechanism introduced in this section is the core design today for fusing information from different positions of the input sequence.[2]

[2]We do not discuss positional embeddings in this section. They are important in practice: self-attention by itself is permutation-equivariant, so a Transformer needs some positional signal, such as learned absolute positional embeddings, sinusoidal embeddings, rotary positional embeddings, or related variants, to distinguish token order. We will add this discussion in a future revision.

Single-head self-attention. Single-head self-attention, denoted by att1h$(\cdot)$, is a function whose input is a sequence of vectors (which are either embeddings or hidden activation) $(h_1^{\text{in}}, \ldots, h_T^{\text{in}})$ and output is also a sequence of vectors (which are always activations) $(h_1^{\text{out}}, \ldots, h_T^{\text{out}})$, where each hidden state is represented as a row vector: $h_t^{\text{in}} \in \mathbb{R}^{1\times d}$ and $h_t^{\text{out}} \in \mathbb{R}^{1\times d_h}$ for all $t \in \{1, \ldots, T\}$. Equivalently, if we stack the input and output vectors into matrices, then att1h maps an input matrix $H^{\text{in}} \in \mathbb{R}^{T\times d}$ to an output matrix $H^{\text{out}} \in \mathbb{R}^{T\times d_h}$. The parameters of the attention layer are $W^Q, W^K, W^V \in \mathbb{R}^{d\times d_h}$, which are often called the query, key, and value projection matrices.

The first step of single-head self-attention linearly transforms the input vectors into queries, keys, and values by multiplying by the query, key and value projection matrices. Intuitively, we can think of the query vector as asking "what information am I looking for?", the key vector as advertising "what information I have?", and the value vector as providing the actual information to be passed forward if the query and key match.

$$q_1 = h_1^{\text{in}} W^Q \ , \ldots, \ q_T = h_T^{\text{in}} W^Q, \tag{17.14}$$

$$k_1 = h_1^{\text{in}} W^K \ , \ldots, \ k_T = h_T^{\text{in}} W^K, \tag{17.15}$$

$$v_1 = h_1^{\text{in}} W^V \ , \ldots, \ v_T = h_T^{\text{in}} W^V \, . \tag{17.16}$$

Then, for each position t in the sequence, we compute the inner product of its query with the keys of all the tokens $q_t k_1^\top, \ldots, q_t k_T^\top$. Without any masking (a concept that we will introduce later), the attention scores $p_{t,1}, \ldots, p_{t,T}$ are defined by applying the softmax function to these inner products.

$$p_{t,1}, \ldots, p_{t,T} = \text{softmax}(q_t k_1^\top / c, \ldots, q_t k_T^\top / c) \, . \tag{17.17}$$

where c is a scaling factor. The inner product between q_t and k_j here are often referred to as "attention", because it intuitively tries to figure out how much the output of token t should depend on, or pay attention to, the token j. The total attention is normalized to sum 1 by the softmax.

In Vaswani et al. [2017], $c = \sqrt{d_h}$. A simple intuition for this scaling is as follows. Suppose that at initialization the coordinates of the query vector q and key vector k behave like independent random variables with mean zero and variance 1 (or, more generally, $O(1)$). Then their dot product

$$q \cdot k = \sum_{i=1}^{d_h} q_i k_i$$

has mean zero and variance d_h up to constants. Without scaling, the attention logits would therefore grow with the head dimension, which will make the magnitude of the logits too large for large d_h and as a result the attention score into the saturated regime, which results in very small gradients. Dividing by $\sqrt{d_h}$ keeps the variance of the logits on the order of one. This is only a heuristic, since the query and key vectors are not generally independent in an actual Transformer. The output of the self-attention at position t is the linear combination of value vectors with attention scores as the coefficient,

$$h_t^{\text{out}} = \sum_{j=1}^{T} p_{t,j} v_j \,. \tag{17.18}$$

In this way, self-attention mixes the values vectors across positions according to the attention scores.

The same computation can be written compactly in matrix form. Stack the query, key, and value row vectors into matrices $Q, K, V \in \mathbb{R}^{T \times d_h}$ as

$$Q = \begin{bmatrix} q_1 \\ \vdots \\ q_T \end{bmatrix} = H^{\text{in}} W^Q, \qquad K = \begin{bmatrix} k_1 \\ \vdots \\ k_T \end{bmatrix} = H^{\text{in}} W^K, \qquad V = \begin{bmatrix} v_1 \\ \vdots \\ v_T \end{bmatrix} = H^{\text{in}} W^V. \tag{17.19}$$

Thus the t-th rows of Q, K, V are q_t, k_t, v_t. Without any masking (which will be introduced a bit later), single-head attention can be written in matrix form as follows,

$$H^{\text{out}} = \text{softmax}_{\text{row}}\big(QK^\top/c\big)\, V, \tag{17.20}$$

The matrix form is not a new model, but the same per-position rule written so all pairwise query-key comparisons can be computed in parallel. This is one reason attention fits modern accelerators well despite the quadratic number of pairwise scores. where $\text{softmax}_{\text{row}}$ applies softmax separately to each row (corresponding to each query vector). One can verify that the t-th row of the matrix $\text{softmax}_{\text{row}}\big(QK^\top/c\big)$ is exactly the attention scores $p_{t,1}, \ldots, p_{t,T}$ defined in (17.17), and thus the t-th row of H^{out} is equal to the h_t^{out} defined in (17.18).

Masking. In an autoregressive Transformer, the final output at position t, that is $f_\theta(x_0, \ldots, x_t)$ defined in Section 17.2, shouldn't depend on any x_j with $j > t$. As we will see in later part where we combine attention with

MLP into a Transformer, the autoregressive property corresponds to the requirement for the attention module that the output vector at position t should not depend on any input vector at position $j > t$. This is achieved by modifying the definition of the attention by introducing an attention mask $M \in \mathbb{R}^{T\times T}$ before the softmax operation, The mask is what lets us train on a full sequence in parallel while preserving the left-to-right prediction rule. Every position can be processed in the same forward pass, but the mask prevents position t from using tokens that would not be available when predicting the next token autoregressively.

$$H^{\text{out}} = \text{softmax}_{\text{row}}\big(QK^\top/c + M\big)\, V. \tag{17.21}$$

The most common mask is called causal mask, which is precisely for achieving the autoregressive property: $M_{t,j} = 0$ for $j \le t$ and $M_{t,j} = -\infty$ for $j > t$. The $-\infty$ entries make the corresponding softmax probabilities exactly zero, so each output hidden state depends only on the current and previous positions. Concretely, one can verify that for the t-th position or t-th row, the logits before attention are

$$q_t k_1^\top/c, \ldots, q_t k_t^\top/c, -\infty, -\infty, \cdots, -\infty. \tag{17.22}$$

As a result, the attention scores are

$$p_{t,1}, \ldots, p_{t,T} = \text{softmax}(q_t k_1^\top/c, \ldots, q_t k_t^\top/c, -\infty, -\infty, \cdots, -\infty), \tag{17.23}$$

so $p_{t,j} = 0$ for all $j > t$. The final output at position t is

$$h_t^{\text{out}} = \sum_{j=1}^{t} p_{t,j} v_j, \tag{17.24}$$

which doesn't depend on any information after the t-th step. Attention masks are useful for other purposes such as ignoring padding tokens in batched sequences, or imposing block-structured visibility constraints when different parts of the input should not attend to one another. We will discuss these use cases in later sections when they are needed.

(Multi-head) Attention. One attention layer has multiple attention heads in parallel by combining many single-head attention modules. Concretely, the output of attention is the linear transformation of the concatenation of the multiple single-head self-attention outputs. Suppose we have n_h attention heads, or more precisely, n_h single-head attention modules that are

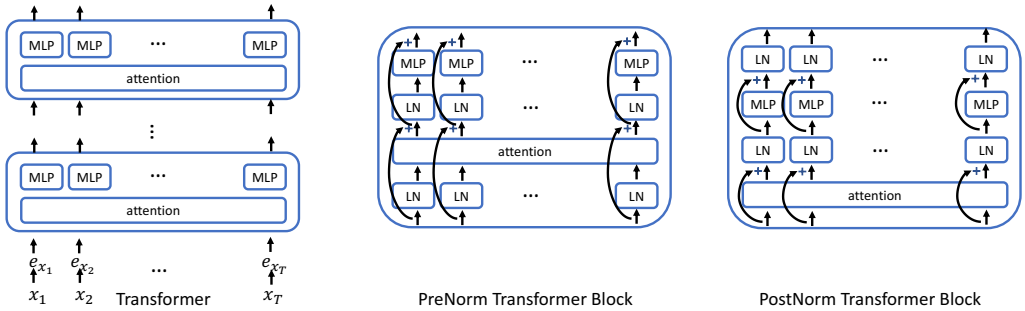


Figure 17.3: The structure of Transformers and Transformer blocks.

denoted by $\text{att1h}_1, \cdots, \text{att1h}_{n_h}$. In practice, people choose n_h and d_h satisfying $n_h \cdot d_h = d$. Recall that the outputs of these single-head attention modules are $\text{att1h}_1(h_1, \ldots, h_T), \cdots, \text{att1h}_{n_h}(h_1, \ldots, h_T)$, each of which is a sequence of vectors. Let $\text{att1h}_j(h_1, \ldots, h_T)_t$ be the t-th vector in $\text{att1h}_j(h_1, \ldots, h_T)$, and let z_t be the concatenation of $\text{att1h}_1(h_1, \ldots, h_T)_t, \cdots, \text{att1h}_{n_h}(h_1, \ldots, h_T)_t$ which is viewed as an $n_h \cdot d_h$ dimensional row vector. Multi-head self-attention introduces an additional parameter, the output weight matrix $W^O \in \mathbb{R}^{(n_h \cdot d_h) \times d}$, and applies it on z_t to get the t-th output of the multi-head attention. More precisely,

$$\text{att}(h_1, \ldots, h_T) = (z_1 W^O, \cdots, z_T W^O). \tag{17.25}$$

For clarity, the attention layer with n_h heads takes in a sequence of T vectors of dimension d and outputs a sequence of T vectors of dimension d. Multiple heads let the layer run several attention rules in parallel. Different heads can specialize in different kinds of token interactions, and the output projection then recombines these separate views into one d-dimensional representation at each position.

Auto-regressive Generation and KV cache Equation (17.21) is the model behavior when all the tokens are available. This is an operation that is needed in (1) training the model where the tokens are given (from the pre-training datasets or self-generated in previous iteration of the model as in RL), and we are interested in computing the likelihood of the current model weights to generate the given sequence of tokens, or (2) the sequence of tokens is users' prompts.

To generate new tokens from the model, one needs to generate each of x_t one-by-one, and then feed the generated results back to the transformer as

shown in (17.7). Note that the computations in (17.7) share prefixes. For example, $f_\theta(x_0, \cdots, x_j)$ and $f_\theta(x_0, \cdots, x_k)$ for $j < k$ both involve computing the keys and values for positions up to j, so these computations can be cached and do not require recomputation. This collection of cached key and value vectors is referred to as the KV-cache. In particular, at time t, the running KV-cache is the collection of all keys and values up to t,

$$K_{1:t} = \begin{bmatrix} k_1 \\ \vdots \\ k_t \end{bmatrix}, \qquad V_{1:t} = \begin{bmatrix} v_1 \\ \vdots \\ v_t \end{bmatrix}, \tag{17.26}$$

and the attention output at the t-th position can be computed as

$$h_t^{\text{out}} = \text{softmax}\left(\frac{q_t K_{1:t}^\top}{c}\right) V_{1:t}. \tag{17.27}$$

As t increments, the KV-cache is appended with new rows k_{t+1}, v_{t+1}, respectively, but the previously cached keys and values do not need to be recomputed. This avoids recomputing keys and values for the whole prefix at every decoding step. We note that the cache above is only for a single attention head, and for standard attention with multiple heads, each head involves a collection of keys and values to be saved in GPU memory.

Transformer. Transformers are largely compositions of alternating applications of attention and MLPs, where MLPs are applied on each individual position (with shared weights) separately, as shown in Figure 17.3.

There are some detailed options on how to combine these. A layer in transformer often refers to a combination of attention and MLP layer. Let $(h_1^\ell, \ldots, h_T^\ell)$ denote the row-vector hidden states entering layer ℓ, where $h_t^\ell \in \mathbb{R}^{1\times d}$, and let $(h_1^{\ell+1}, \ldots, h_T^{\ell+1})$ denote the hidden states leaving the layer. There are two kinds of mainstream structures: *PostNorm* and *PreNorm*. In PreNorm, the layer normalization is applied before the MLP and the self-attention.

$$r_t = h_t^\ell + [\text{att}(\text{LN}(h_1^\ell), \ldots, \text{LN}(h_T^\ell))]_t \tag{17.28}$$

$$h_t^{\ell+1} = r_t + \text{MLP}(\text{LN}(r_t))\,. \tag{17.29}$$

In PostNorm, the layer normalization is applied after the residual connection.

$$r_t = \mathrm{LN}(h_t^\ell + [\mathrm{att}(h_1^\ell, \ldots, h_T^\ell)]_t) \tag{17.30}$$

$$h_t^{\ell+1} = \mathrm{LN}(r_t + \mathrm{MLP}(r_t)) \,. \tag{17.31}$$

See Figure 17.3 for an illustration of PreNorm and PostNorm Transformer block structures. Modern large language models usually use PreNorm structure, and with RMSNorm instead of layer norm.

Memory and compute. We discuss the memory and compute costs of attention.

Compute. In training and prompt prefilling, when all tokens are given, each attention head costs $O(T^2 d_h)$ arithmetic operations for the masked attention computation in Equation (17.21). For multi-head attention with n_h heads, the n_h copies of Equation (17.21) cost $O(T^2 d_h n_h)$ operations, while the output projection in Equation (17.25) costs $O(T n_h d_h d)$ operations and is smaller and not the dominating term for large T. The important takeaway is that the attention computation is quadratic in T, which is a main source of difficulty for long-context settings where T is larger than 10K, and sometimes up to 1M or more.

During generation, the total number of attention operations needed to generate T tokens is of the same order, but generation must be done sequentially token by token. This often makes it harder to saturate GPU parallelism and achieve an optimal level of utilization.

Memory. The GPU memory usage of attention depends on exactly how the computation is implemented. In training and prefilling, a naive implementation of Equation (17.21) costs T^2 memory per head because the attention-score matrix $QK^\top$ has T^2 entries. However, methods such as FlashAttention [Dao et al., 2022] can use memory linear in T for the attention computation, essentially saving Q, K, and V and recomputing or streaming the attention scores through an equivalent sequence of operations, rather than explicitly materializing the full $QK^\top$ matrix.

In generation and decoding, the KV cache needs to be saved in GPU memory, which costs $O(T d_h)$ per head for a length-T context, unless one is willing to move part of the KV cache to CPU memory, which is comparatively slow.

In the next section, we discuss ways to improve efficiency by changing the attention architecture.

17.4 Variants of Attention

GQA, MQA, and sliding window attention. Several attention variants reduce the memory size of the KV cache. Multi-query attention (MQA) keeps multiple query heads but shares a single key head and a single value head across them [Shazeer, 2019]. Grouped-query attention (GQA) is an intermediate design: the query heads are partitioned into groups, and each group shares one key head and one value head [Ainslie et al., 2023]. Thus, ordinary multi-head attention, GQA, and MQA form a spectrum trading off model quality and decoding efficiency, with the KV cache size decreasing as fewer key-value heads are used.

Mathematically, GQA means that for the t-th position, we have n_h queries, denoted by $q_{t,1}, \ldots, q_{t,n_h}$, and n_g keys and values, denoted by $k_{t,1}, \cdots, k_{t,n_g}$ and $v_{t,1}, \cdots, v_{t,n_g}$, where n_g divides n_h. Let $\tau = n_h/n_g$ be the number of query heads assigned to each key/value head. For query head $j \in \{1, \ldots, n_h\}$, define the group index

$$g(j) = \left\lfloor \frac{j-1}{\tau} \right\rfloor + 1 \in \{1, \ldots, n_g\}. \tag{17.32}$$

Thus the query head j uses the key and value head $g(j)$. Let Q_j be the matrix whose t-th row is $q_{t,j}$ for $j \in \{1, \ldots, n_h\}$. For $s \in \{1, \ldots, n_g\}$, let K_s, V_s be the matrices whose t-th rows are $k_{t,s}$ and $v_{t,s}$, respectively.

$$Q_j = \begin{bmatrix} q_{1,j} \\ \vdots \\ q_{T,j} \end{bmatrix}, \qquad K_s = \begin{bmatrix} k_{1,s} \\ \vdots \\ k_{T,s} \end{bmatrix}, \qquad V_s = \begin{bmatrix} v_{1,s} \\ \vdots \\ v_{T,s} \end{bmatrix}. \tag{17.33}$$

The j-th output head, denoted by H_j^{out}, is then computed by

$$H_j^{\text{out}} = \text{softmax}_{\text{row}}\left(\frac{Q_j K_{g(j)}^\top}{c} + M \right) V_{g(j)}\,, \tag{17.34}$$

where M is the attention mask, such as the causal mask in autoregressive decoding. Standard multi-head attention is the special case $n_g = n_h$, while MQA is the special case $n_g = 1$.

Another approach is *sliding-window attention*, where each token only attends to the most recent w tokens instead of the entire prefix. This corresponds to a causal local attention mask

$$M_{t,j} = \begin{cases} 0, & \max(1, t-w+1) \le j \le t, \\ -\infty, & \text{otherwise}, \end{cases} \tag{17.35}$$

so keys and values older than the window do not affect the current output and need not remain in the active KV cache.

Query-key normalization. Another attention variant is query-key normalization, or QK-Norm [Henry et al., 2020]. In standard attention, the attention logits are dot products $q_t k_j^\top / c$, so their magnitude depends on the norms of the query and key vectors. If these norms become large, the softmax distribution can become very sharp or saturated. QK-Norm instead normalizes each query and key vector before taking their dot product, for example replacing $q_t k_j^\top$ with a scaled cosine similarity $\text{RMSNorm}(q_t)\text{RMSNorm}(k_j)^\top$ for RMSNorm defined in Equation (7.51). The motivation is to control the scale of attention logits and improve training stability while still letting the model learn the overall sharpness of the attention distribution. This approach has been adopted in several recent open-weight models, for example, Qwen3 and OLMo2 [Yang et al., 2025, OLMo Team et al., 2024].

17.5 Mixture-of-Experts Layers

A different way to increase the size of a Transformer is to make some layers *sparse*. The most common example is a Mixture-of-Experts (MoE) layer. In an ordinary Transformer block, the MLP is a dense module: every token is processed by the same MLP, so every token uses the same parameters and incurs the same MLP computation. In an MoE block, the model instead has several MLPs, called *experts*, and a small routing network decides which experts should process each token.

Concretely, suppose the layer has experts $E_1, \ldots, E_m$, where each E_s is an MLP. Given the hidden state h_t of token t, a router computes routing weights

$$r_t = \sigma(h_t W^R) \in \mathbb{R}^m,$$

where σ is a routing function, commonly a softmax over experts or a sigmoid applied to each expert score. The model then chooses a small set S_t of experts, often of a constant size k, and combines their outputs as

$$\text{MoE}(h_t) = \sum_{s \in S_t} \alpha_{t,s} E_s(h_t), \qquad \alpha_{t,s} = \frac{r_{t,s}}{\sum_{j \in S_t} r_{t,j}}.$$

For example, DeepSeek-V3 uses 256 routed experts in each MoE layer and activates the top 8 routed experts for each token, in addition to one shared expert [DeepSeek-AI, 2024]. A shared expert is applied to every token, so it

acts as a common feedforward path while the routed experts provide token-dependent specialization. Thus each token only activates a few experts, even though the layer contains many expert parameters. This is the main appeal of MoE: it increases the total parameter count and representational capacity of the model without increasing the per-token computation by the same factor. One can think of the router as choosing which specialized MLPs are most relevant for the current token and context. MoE layers are usually used to replace some or all of the dense MLP layers in a Transformer block.

17.6 In-context learning

In-context learning is mostly used for few-shot settings where we have a few labeled examples $(x^{(1)}_{\text{task}}, y^{(1)}_{\text{task}}), \cdots, (x^{(n_{\text{task}})}_{\text{task}}, y^{(n_{\text{task}})}_{\text{task}})$. Given a test example x_{test}, we construct a prompt $(x_1, \cdots, x_T)$ by concatenating the labeled examples and the test example in some format. For example, we may construct the prompt as follows

$$
\begin{aligned}
x_1, \cdots, x_T \quad = \quad & \text{“Q: } 2 \sim 3 = \text{?} && x^{(1)}_{\text{task}} \\
& \text{A: } 5 && y^{(1)}_{\text{task}} \\
& \text{Q: } 6 \sim 7 = \text{?} && x^{(2)}_{\text{task}} \\
& \text{A: } 13 && y^{(2)}_{\text{task}} \\
& \cdots \\
& \text{Q: } 15 \sim 2 = \text{?”} && x_{\text{test}}
\end{aligned}
$$

Then, we let the pretrained model generate the most likely $x_{T+1}, x_{T+2}, \cdots$. In this case, if the model can “learn” that the symbol $\sim$ means addition from the few examples, we will obtain the following which suggests the answer is 17.

$$x_{T+1}, x_{T+2}, \cdots = \text{“A: } 17\text{”}.$$

The same idea can be used as more practical prompts. For instance, suppose we want a language model to classify customer-support messages while also returning the answer in a fixed machine-readable format. We may

give the model examples such as

"Task: Classify each message as {billing, technical, account}. Return JSON.
Message: I was charged twice for my subscription.
Output: {"label": "billing"}
Message: The app crashes when I upload a file.
Output: {"label": "technical"}
Message: Please change the email address on my profile.
Output: {"label": "account"}
Message: I cannot reset my password.
Output: "

The desired continuation is a JSON object such as `{"label": "account"}`. In this example, the in-context demonstrations serve two roles: they identify the task and label space, and they teach the model an output format. More elaborate prompts can also demonstrate a workflow, such as first extracting relevant facts and then producing a final answer. In-context learning was popularized as a capability of large language models by Brown et al. [2020]; subsequent work studies what information the demonstrations provide, such as the input distribution, label space, and formatting conventions [Min et al., 2022], as well as how demonstrations of intermediate reasoning steps can improve some reasoning tasks [Wei et al., 2022].

17.7 Zero-shot learning / prompting

For autoregressive language models, the most lightweight form of adaptation is prompting: we change the text given to the model, but do not change the model parameters. The in-context learning examples in the previous section are one form of prompting. In the zero-shot setting, there are no input-output examples from the downstream task, so the prompt usually describes the task in natural language and asks the model to produce an answer in a specified format. For example, we may ask

$$
\begin{aligned}
x_{\text{task}} &= (x_{\text{task},1}, \cdots, x_{\text{task},T}) \\
&= \text{"Question: Is the speed of light a universal constant?} \\
&\quad \text{Answer yes or no."}
\end{aligned}
$$

and then decode a continuation from $p_\theta(\cdot \mid x_{\text{task}})$. If the model continues with "No," it has produced the intended label for this prompt. More generally, the output may be decoded greedily, sampled, constrained to a set of labels, or parsed from a longer generated answer. Prompting is attractive because it requires no training data or gradient updates, but its performance can depend on the pretrained model and on the wording of the prompt.

17.8 Supervised Finetuning (SFT)

Finetuning instead adapts the model by updating parameters on downstream examples. For autoregressive language models, this usually does not require adding a new prediction head. We can format each supervised example as a prompt-completion pair $(x^{(i)}, y^{(i)})$, where $x^{(i)}$ consists of a sequence of tokens $x_1^{(i)}, \ldots, x_{L_i}^{(i)})$ and $y^{(i)}$ consists of a sequence of tokens $y_1^{(i)}, \cdots, y_{T_i}^{(i)}$. We initialize θ at the pretrained parameters $\hat{\theta}$, and continue minimizing the conditional next-token loss

$$\min_\theta \frac{1}{n} \sum_{i=1}^{n} \left[-\sum_{t=1}^{T_i} \log p_\theta\left(y_t^{(i)} \mid x^{(i)}, y_{<t}^{(i)}\right) \right]. \tag{17.36}$$

For a classification task, for instance, $y^{(i)}$ may simply be the text of the class label or a short JSON object containing the label. Here $x^{(i)}$ is the question, instruction (see paragraph below), or user prompt, while $y^{(i)}$ is the desired answer or assistant response. A common implementation concatenates the prompt and answer into one causal-LM sequence

$$z^{(i)} = (x_1^{(i)}, \ldots, x_{L_i}^{(i)}, y_1^{(i)}, \ldots, y_{T_i}^{(i)}),$$

but applies the next-token loss only to the answer positions. Equivalently, for a minibatch $\mathcal{B}$, one uses a loss mask $m_r^{(i)}$ with $m_r^{(i)} = 0$ on prompt tokens and $m_r^{(i)} = 1$ on answer tokens, and minimizes the average loss over all answer-token positions in the minibatch,

$$-\frac{1}{\sum_{i\in\mathcal{B}} \sum_{r=1}^{L_i+T_i} m_r^{(i)}} \sum_{i\in\mathcal{B}} \sum_{r=1}^{L_i+T_i} m_r^{(i)} \log p_\theta\left(z_r^{(i)} \mid z_{<r}^{(i)}\right). \tag{17.37}$$

This loss mask is separate from the attention mask: the usual causal attention mask is still used, so answer tokens can attend to the prompt and to previous answer tokens, but the prompt tokens themselves do not contribute to the SFT objective.

Instruction tuning. Instruction tuning is an important special case of finetuning. Here an *instruction* means a natural-language task description, such as "summarize the following paragraph in one sentence," "classify this customer message as billing, technical, or account-related," or "write a Python function that sorts a list." Instead of training on only one downstream task, we collect many tasks written as natural-language instructions with desired responses, and finetune the model on the resulting mixture [Wei et al., 2021, Ouyang et al., 2022]. The goal is not merely to fit those training tasks, but to teach the model the convention that an instruction in the prompt should be followed. This is why instruction-tuned models often become much better zero-shot assistants: at test time, the user can describe a new task in words, and the model has been trained to treat such descriptions as executable instructions.